

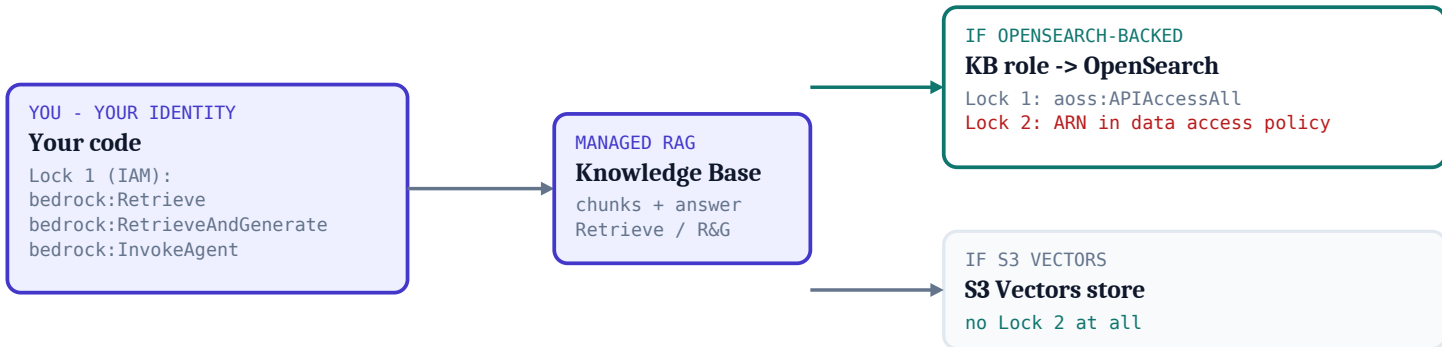
TravelMind - Bedrock runtime lab

Three ways to call a Knowledge Base from code: `retrieve`, `retrieve_and_generate`, `invoke_agent`

account 452203592848 | region us-east-1 | client = bedrock-agent-runtime

THE TWO LOCKS, IN THIS LAB

Who clears which lock



Your code clears exactly one lock: IAM on you. The Knowledge Base clears its own two locks on your behalf when it is OpenSearch-backed. An S3 Vectors KB has no Lock 2, which is why the class uses it.

403 on your call = Lock 1 on you (a missing `bedrock:*` action). 403 with "no index access" during the KB's lookup = Lock 2 on the KB role.

RUN IT

Pick one environment

VS Code	<code>python -m venv .venv -> activate -> pip install boto3</code> . Set creds with <code>aws configure</code> (or env keys + <code>AWS_DEFAULT_REGION=us-east-1</code>). Select <code>.venv</code> as the interpreter. Fill the IDs, run.
Colab	<code>!pip install boto3</code> . Creds via Colab secrets -> <code>os.environ["AWS_ACCESS_KEY_ID"]</code> etc., set <code>AWS_DEFAULT_REGION</code> . Fill the IDs, run.
CloudShell	<code>boto3</code> and creds are already present. Fill the IDs, then: <code>python bedrock_runtime_lab.py</code>

1 - RETRIEVE

Raw chunks, you build the prompt

```

rt = boto3.client("bedrock-agent-runtime", region_name="us-east-1")

r = rt.retrieve(
    knowledgeBaseId=KB_ID,
    retrievalQuery={"text": query},
    retrievalConfiguration={"vectorSearchConfiguration":{"numberOfResults":5}})
for ch in r["retrievalResults"]:
    print(ch["score"], ch["content"]["text"][:200])
  
```

Production: scope `bedrock:Retrieve` to the KB ARN, not `""`; tune `numberOfResults` to your context budget; never log raw chunk text if it can hold PII.

2 - RETRIEVE_AND_GENERATE

Chunks + a finished answer in one call

```
r = rt.retrieve_and_generate(  
    input={"text": query},  
    retrieveAndGenerateConfiguration={  
        "type": "KNOWLEDGE_BASE",  
        "knowledgeBaseConfiguration": {  
            "knowledgeBaseId": KB_ID,  
            "modelArn": MODEL_ARN}})    # us. inference-profile prefix is mandatory  
print(r["output"]["text"])             # pass r["sessionId"] back to continue a turn
```

Production: needs `bedrock:RetrieveAndGenerate` + `bedrock:InvokeModel` on the inference profile; the region must match the KB; cross-region inference adds \$0.02/GB transfer; catch `ClientError` and back off.

3 - INVOKE_AGENT

Full Agent-Builder agent (event stream)

```
resp = rt.invoke_agent(  
    agentId=AGENT_ID, agentAliasId=AGENT_ALIAS_ID,  
    sessionId=str(uuid.uuid4()), inputText=text)  
for event in resp["completion"]:    # EventStream - you MUST iterate it  
    if "chunk" in event:  
        print(event["chunk"]["bytes"].decode(), end="")  
# enableTrace=True surfaces the agent reasoning / KB lookup / tool steps
```

Production: use a versioned alias (never the draft / TSTALIASID); your identity only needs `bedrock:InvokeAgent`; the agent runs as its own service role; handle failures mid-stream; turn trace off.

WHAT CHANGES IN PRODUCTION

Read before you ship any of the above

- IAM roles, not access keys. A Lambda or ECS task role replaces aws configure; drop the keys.
- No hardcoded region, IDs, or secrets. Read them from env / SSM Parameter Store / Secrets Manager.
- Least privilege. Scope to the exact KB ARN and the agent-alias ARN, not "*".
- Use a versioned agent alias, never the draft / TSTALIASID.
- Retries and errors. Adaptive retry Config; catch `ClientError`; exponential backoff on throttling.
- Region must match the KB. Cross-region inference adds \$0.02/GB data transfer.
- Observability. CloudWatch logs and metrics; log the requestId; enableTrace only in staging.
- Do not log raw user content or retrieved chunks if they can contain PII.